



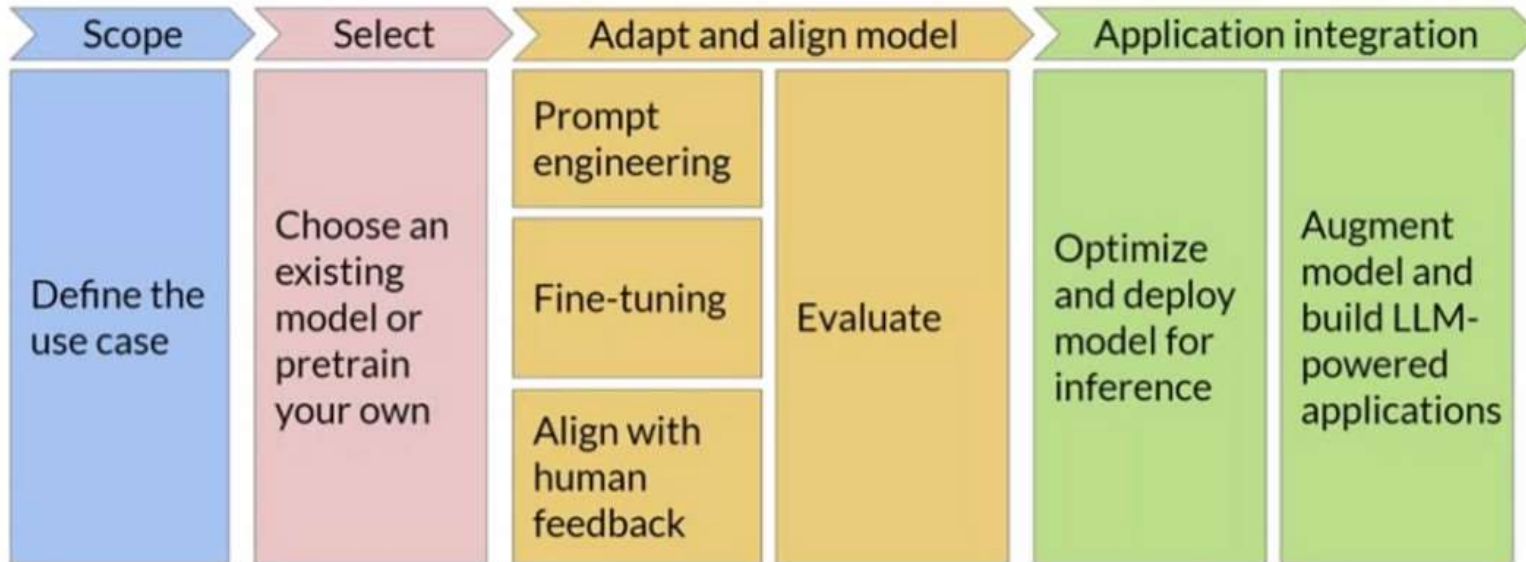
UE23CS342BA9 - Generative AI and its Applications

Generative AI Lifecycle

Dr. UMA D

Professor

Department of Computer Science and Engineering



The basic Project lifecycle of a Generative AI deals with 4 core principles

- Scope : define the problem statement you want to solve using LLM to determine how it should word

- Select : Choose a model or pretrain your — existing one or train from scratch

Adapt and align model : develop and align model to design with

- Prompts, fine tune and evaluate for best output

Application integration : optimize and deploy models for inference, build LLM powered applications



Step 1: Defining the Scope

- Clearly scope the LLM task for:
 - Specific tasks: Named Entity Recognition, Classification, Summarization.
 - Broad tasks: Multi-purpose chatbots, creative text generation.
- Fine-tuning Impact: The more specific the task, the less compute and training data you'll need.

Practical Considerations:

- Will the task evolve (e.g., domain adaptation)?
- How will performance be measured (accuracy, relevance)?

Tip: Use small-scale models for prototyping before committing to fine-tuning larger models.



Step 2: Choose the Base Model

- Base Model Selection for Fine-Tuning:

- Popular choices: GPT-3.5, T5, BERT, LLaMA, Falcon.
- When to Use an Existing Model:
 - Faster development and cost-efficient.

- Pre-trained on general knowledge.

When to Pre-Train from Scratch:

- Highly specialized domains with insufficient pre-trained data.

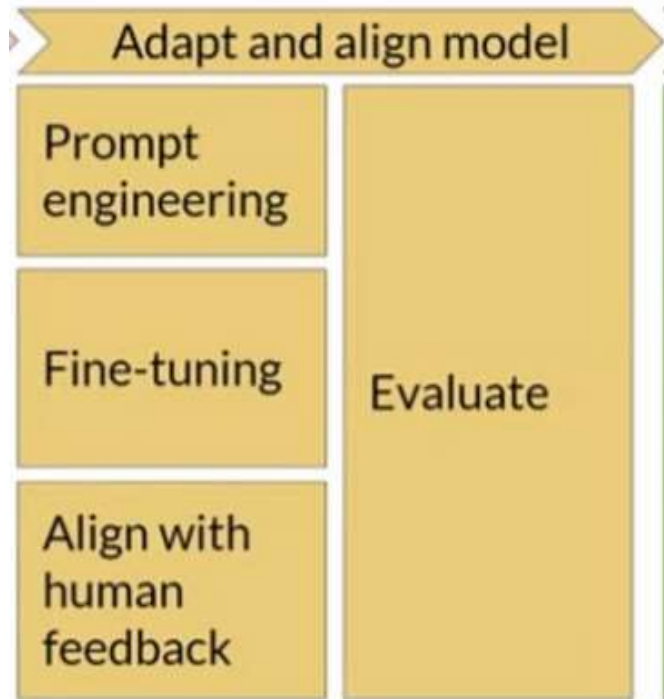
Fine-Tuning Base Models:

Choose smaller models for cost-sensitive applications.

Leverage models with open weights for custom fine-tuning (e.g., Hugging Face Transformers).

Select

Choose an
existing
model or
pretrain
your own



Step 3: Adapt and Align the Model

1. Prompt Engineering

- Modify input prompts for improved responses (zero-shot, one-shot, few-shot). Faster and requires no retraining, but limited for domain-specific tasks.

Step 3: Adapt and Align the Model

2. Fine-Tuning

- Fine-tuning is supervised learning: Adapt a pre-trained model to your task.

Requires:

- Task-specific labeled dataset.
- Compute resources for training.

Types of Fine-Tuning:

- Full Fine-Tuning: Updates all weights (expensive but powerful).
- Parameter-Efficient Fine-Tuning (PEFT): Updates fewer parameters to save cost.

Methods: LoRA, Adapters, Prefix-Tuning.

Use Cases for Fine-Tuning:

Domain specific tasks: legal, medical, or scientific text

Step 3: Adapt and Align the Model

2. Fine-Tuning

- Fine-tuning is supervised learning: Adapt a pre-trained model to your task.

Requires:

- Task-specific labeled dataset.
- Compute resources for training.

Types of Fine-Tuning:

- Full Fine-Tuning: Updates all weights (expensive but powerful).
- Parameter-Efficient Fine-Tuning (PEFT): Updates fewer parameters to save cost.

Methods: LoRA, Adapters, Prefix-Tuning.

Use Cases for Fine-Tuning:

Domain specific tasks: legal, medical, or scientific text

Step 3: Adapt and Align the Model

3. Align with Human Feedback

- RLHF (Reinforcement Learning with Human Feedback): Fine-tune models to align outputs with human preferences.
- Used for safety, quality, and usability improvements.

Example: OpenAI's GPT models use RLHF for conversational alignment.

4. Evaluation

- Use benchmarks and metrics to measure performance:
- Metrics: Perplexity, Accuracy, BLEU, ROUGE.

Task-specific evaluation: Exact match, F1 Score (e.g., for QA tasks).

Iterative Process: Evaluate → Fine-tune → Optimize.

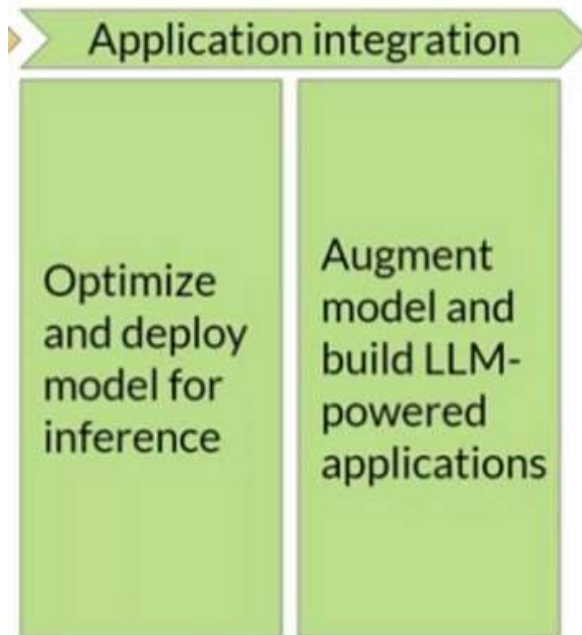
Fine-Tuning with PEFT Techniques

- Reduce compute cost while maintaining performance.
- Popular PEFT Methods:
 - LoRA (Low-Rank Adaptation): Updates low-rank matrices instead of all weights.
 - Adapters: Introduce small bottleneck layers into the model.
 - Prefix-Tuning: Tune task-relevant prefix embeddings while freezing the model.

Comparison:

Method	Parameters Updated	Use Case
LoRA	Low	Domain-specific fine-tuning
Adapters	Low	Modular and efficient tuning
Prefix-Tuning	Very Low	Fast adaptation for NLP tasks

Step 4: Deploy and Optimize the Model



- 1. Optimize for Inference:
 - Quantization: Reduce model size (e.g., 16-bit → 8-bit precision).
 - Distillation: Use a smaller "student" model that mimics the larger model.
- 2. Model Deployment:
 - Deploy on cloud platforms: AWS, GCP, Azure.
 - Use libraries like Hugging Face Inference APIs or ONNX Runtime.
- 3. Overcoming LLM Limitations:
 - Combine LLMs with tools like:
 - Knowledge bases for factual accuracy.
 - External tools for reasoning (calculators, APIs).

Fine-Tuning Workflow Overview

- Steps to Fine-Tune an LLM:
 - 1. Prepare the Dataset: Curate high-quality labeled data.
 - 2. Select the Base Model: Use a pre-trained model (BERT, GPT-3, LLaMA).
 - 3. Fine-Tune the Model: Choose full fine-tuning or PEFT.
 - 4. Evaluate Performance: Use task-specific metrics.
 - 5. Iterate and Optimize: Improve outputs through retraining.

- The slides/diagrams in this PPT are an **adaptation**, **combination**, and **enhancement** of material from the following resources:
 1. <https://www.deeplearning.ai/courses/generative-ai-with-llms/>
 2. <https://medium.com/@kanikaadik07/generative-ai-project-life-cycle-55ce9092e24a>



THANK
~~YOU~~

Generative AI and
its Applications
Dr. Uma D